DEVOPS CAPSTONE

CampusDesk

Campus IT, organized.

A helpdesk for campus requests, delivered through a tested container and Kubernetes workflow.

T. Abdul Kalam Azad
24BCS10053

APPLICATION

Every request has a clear next step

Students report network, hardware, software and account issues.

The support team sets priority, assigns an owner and resolves tickets.

Search and status filters make the queue easier to manage.

Synthetic demo tickets exercise the same APIs as newly created requests.

TECHNOLOGY

One application, three core layers

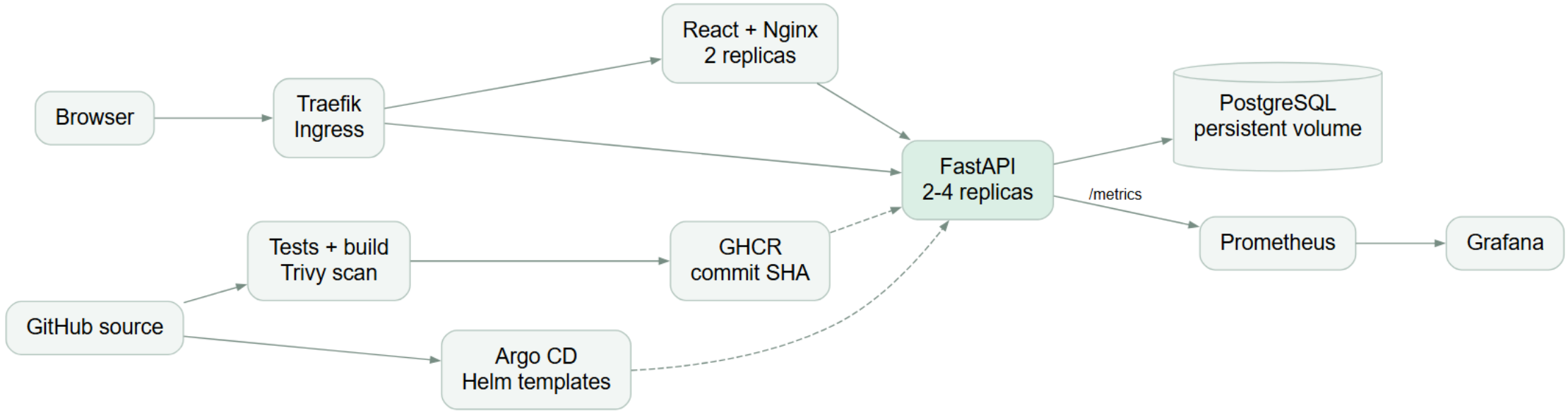
React + Vite + Nginx deliver the responsive service desk.

FastAPI validates requests; SQLAlchemy manages database operations.

PostgreSQL persists tickets; Alembic versions the schema.

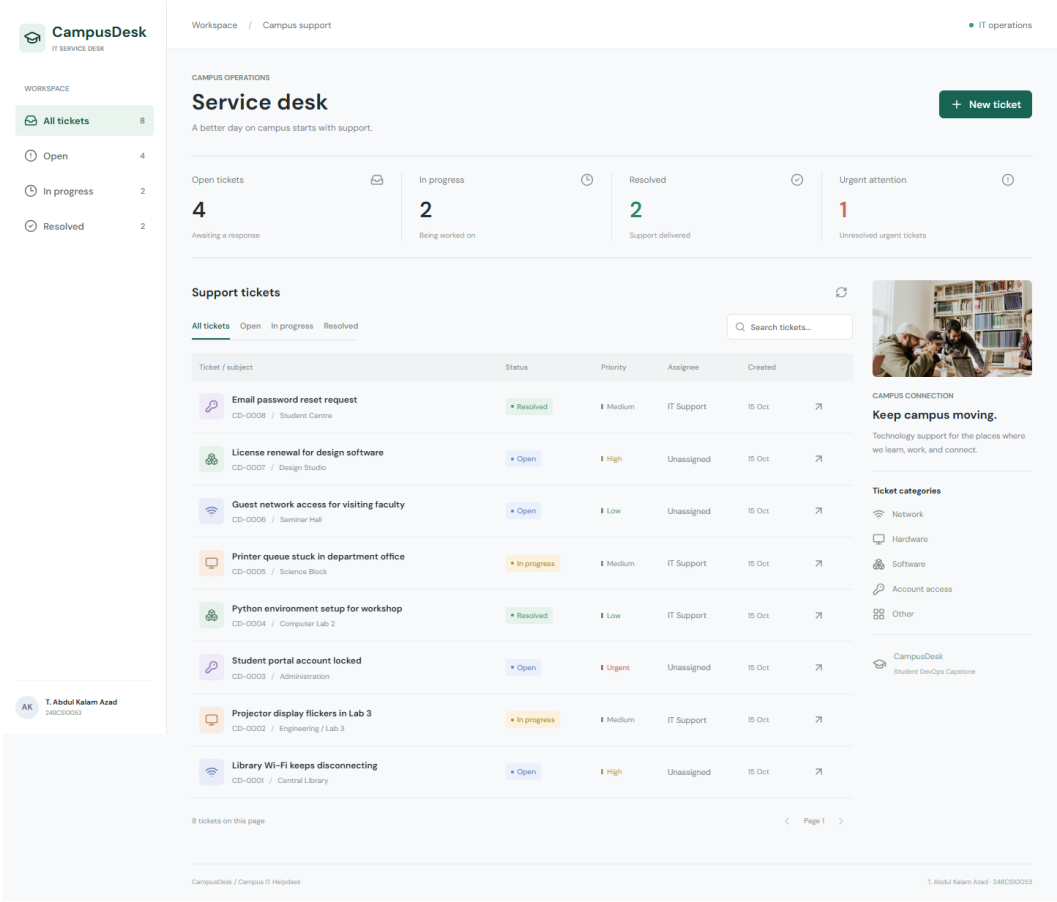
Python tests and Ruff catch regressions before image publication.

Traffic and changes follow defined paths



WORKING APPLICATION

The service desk runs in the browser



Quality gates precede publication

Ruff, database migrations, pytest and the frontend build run first.

Both images are scanned for HIGH and CRITICAL vulnerabilities.

The Helm release is smoke-tested in an ephemeral Kind cluster.

Successful main-branch runs publish images tagged with the commit SHA.

Kubernetes manages application replicas

Helm defines Deployments, Services, Secrets and PostgreSQL storage.

Traefik routes the UI and API through one Ingress hostname.

Readiness checks database access; liveness checks the process.

Metrics Server supports an HPA with a range of two to four API replicas.

Git records the desired deployment

Argo CD reads the CampusDesk Helm chart from the GitHub repository.

Sync compares the committed configuration with the running cluster.

Migration ordering prepares the database before application rollout.

CI promotes scanned image tags; automatic sync deploys that Git revision.

Metrics explain what the service is doing

Prometheus discovers and scrapes each backend replica.

Grafana shows request rate, p95 latency, status codes and availability.

A bounded load generator creates real API traffic for the dashboard.

Backend availability and error-ratio alert rules detect service problems.

AWS infrastructure is defined in code

Two public subnets and two private subnets span availability zones.

A NAT gateway connects private worker nodes to external registries.

EKS, a managed node group and EBS CSI provide the deployment foundation.

AWS run: plan, apply, VPC, EKS, nodes and destroy captured from my ShelfWise configuration.

Failures become evidence for diagnosis

A nonexistent image tag demonstrates ImagePullBackOff and pod events.

A wrong Service selector demonstrates missing endpoints.

A failed database connection makes readiness return HTTP 503.

Image scans guide remediation; successful deployment alone is not security proof.

KEY LEARNINGS

Verification is part of the deliverable

A working app needs tests, repeatable deployment and observable behavior.

Screenshots and reports should show actual runs and identify their environment.

Security gates only help when a failing scan really blocks publication.

Git as the record of deployment makes every release traceable to a commit.

Brief, rubric and submission guide: github.com/Nency-Ravaliya/devsecops-python